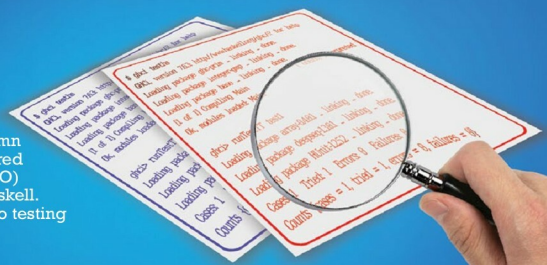
Testing Haskell Programs



Our previous column in this series covered the input/output (IO) functionality of Haskell. We now move on to testing Haskell programs.

HUnit is a unit testing framework available for Haskell. It is similar to JUnit, which is used for the Java programming language. You can install HUnit on Fedora using the following command:

```
$ sudo yum install ghc-HUnit-devel
```

Consider the simple example that follows:

```
import Test.HUnit
test1 = TestCase $ assertEqual "Test equality" 3 (2 + 1)
```

The TestCase is a constructor defined in the Test data type. The definition is as follows:

```
-- Test Definition
-- ===============

-- | The basic structure used to create an annotated tree of
test cases.
data Test
    -- | A single, independent test case composed.
    = TestCase Assertion
    -- | A set of @Test@s sharing the same level in the
hierarchy.
    | TestList [Test]
    -- | A name or description for a subtree of the @Test@s.
    | TestLabel String Test
```

On executing the above code with GHCi, you get the following output:

```
$ ghci test.hs
GHCi, version 7.6.3: http://www.haskell.org/ghc/:? for help
Loading package ghc-prim ... linking ... done.
Loading package integer-gmp ... linking ... done.
Loading package base ... linking ... done.
[1 of 1] Compiling Main             ( one.hs, interpreted )
Ok, modules loaded: Main.

ghci> runTestTT test1
Loading package array-0.4.0.1 ... linking ... done.
Loading package deepseq-1.3.0.1 ... linking ... done.
Loading package HUnit-1.2.5.2 ... linking ... done.
Cases: 1  Tried: 1  Errors: 0  Failures: 0
Counts {cases = 1, tried = 1, errors = 0, failures = 0}
```

You can build a test suite of tests with TestList as shown below:

```
import Test.HUnit

test1 = TestList [ "Test addition" ~: 3 ~=? (2 + 1)
                 , "Test subtraction" ~: 3 ~=? (4 - 1)
                 ]
```

The '~=?' operation is shorthand to assert equality. Its definition is as follows:

```
-- | Shorthand for a test case that asserts equality (with the
expected
--   value on the left-hand side, and the actual value on the
right-hand
--   side).
(~=?) :: (Eq a, Show a) => a     -- ^ The expected value
                        -> a     -- ^ The actual value
                        -> Test
```